DAM
Desarrollo de Aplicaciones Multiplataforma
2º Curso

AD
Acceso a Datos

UD 0
Java
ANEXO

IES BALMIS
Dpto Informática
Curso 2026-2027
Versión 1 (09/2026)

UD2 – Manejo de Ficheros

ÍNDICE

1. Tipos de datos básicos
2. Conversión entre tipos
3. Creación de objetos

1. Tipos de datos básicos

En Java existen varios tipos de datos básicos.

Vamos a crear unos datos de ejemplo en formato primitivo.

Creación de datos básicos en formato primitivo (no disponemos de métodos en estos datos)

```
byte   b; //Entero con 1 byte  => -128 to 127
short  s; //Entero con 2 bytes => -32,768 to 32,767
int    i; //Entero con 4 bytes => -2,147,483,648 a 2,147,483,647
long   l; //Entero con 8 bytes => -9,223,372,036,854,775,808 a 9,223,372,036,854,775,807
float  f; //Decimal con 4 bytes (menos precisión)
double d; //Decimal con 8 bytes (más precisión)
                    // => 1.40129846432481707e-45 to 3.40282346638528860e+38
```

De todas formas, podemos usar los métodos static de las clases de datos.
Por ejemplo:

```
double d = Double.min( 23.45, 45.896 );
```

Vamos a crear unos datos de ejemplo en formato objeto.

Creación de datos básicos en formato Objeto (disponemos de métodos)

```
String  str;
Byte    b;  // Entero con 1 byte  => -128 to 127
Short   s;  // Entero con 2 bytes => -32,768 to 32,767
Integer i;  // Entero con 4 bytes => -2,147,483,648 a 2,147,483,647
Long    l;  // Entero con 8 bytes
                  // => -9,223,372,036,854,775,808 a 9,223,372,036,854,775,807
BigInteger bi;  // Entero dinámico
                  // => Ilimitado: Almacena cualquier valor y ocupa según el número
Float   f;  // Decimal con 4 bytes (menos precisión)
Double  d;  // Decimal con 8 bytes (más precisión)
                  // => 1.40129846432481707e-45 to 3.40282346638528860e+38

        b  = 97;
        s  = 1315;
        i  = 10584789;
        l  = 105847895678L;       // Añadir la L al final
        bi = new BigInteger("54786457895465879415285465453654656313654");
        f  = 35.48f;              // Añadir la f al final
        f  = (float) 35.48;       // Hacer CAST
        d  = 4567894.124578;
```

Estos objetos disponen de métodos.
Por ejemplo:

```
String s = d.toString();
Float  f = d.floatValue();
```

2. Conversión entre tipos

Conversión de datos a String

```
str = String.valueOf(x); // Convierte de cualquier otra clase a String
```

Conversión de String a datos/objeto con valueOf

```
Integer iv = Integer.valueOf("2547");
Double  dv = Double.valueOf("324.145");
```

Conversión de String a datos con parse

```
int iv    = Integer.parseInt("2547");
double dv = Double.parseDouble("324.145");
```

Conversión entre datos mediante CAST, aunque no siempre es posible la conversión

```
// CAST Implícito (De mayor rango a menor)
int num = 10;
double d = num;  // Conversión automática de int a double

// CAST Explícito (De menor rango a mayor)
double d = 9.78;
int num = (int) d;  // Se debe hacer el casting explícito,
                    // ya que se pierde precisión al perder los decimales
System.out.println(num);  // Imprime 9

double d = 35.48;
f = (float) 35.48;  // Se hace el CAST,
                    // aunque en este caso no se pierde
                    // porque el número no es necesario reducirlo/redondearlo
```

3. Creación de objetos

3.1 Constructor

El **constructor** de una clase en Java es un método especial que se llama cuando se crea una instancia (u objeto) de esa clase.

Su propósito principal es inicializar los valores de los atributos del nuevo objeto.

El método del constructor puede estar sobrecargado por lo que podría recibir varios tipos de argumentos como parámetros.

El nombre del método es el mismo que el de la clase y se llama con new.

Constructor

```
str = new String("1234");

byte[] ba = {'4','5','6','7'};
str = new String(ba);

// En tipos de datos básicos se puede simplificar con:
str = "1234";
```

3.2 Parser

Un **Parser** es una estructura de código utilizada para analizar datos, generalmente en formato de texto, y convertirlos en un formato más manejable para el programa, como datos u objetos Java.

Parser

```
// Método    => .parse

int    x = Integer.parseInt("12");
double c = Double.parseDouble("12");
```

3.3 Factory

El **Factory Pattern** (Patrón de Fábrica) se utiliza para crear objetos sin exponer el proceso de creación al cliente, delegando la responsabilidad de instanciar objetos a una clase fábrica.

Factory

```
// Método    => .getInstance()

// Ejemplo   => NumberFormat
NumberFormat formatoNumero = NumberFormat.getInstance();
double numero = 12345.6789;
System.out.println("Número: " + formatoNumero.format(numero));

// Ejemplo   => Calendar
Calendar c = Calendar.getInstance();
String dia, mes, annio;
dia  = Integer.toString(c.get(Calendar.DATE));
mes  = Integer.toString(c.get(Calendar.MONTH));
anyo = Integer.toString(c.get(Calendar.YEAR));
```

3.4 Builder

El **Builder Pattern** se utiliza para construir objetos complejos paso a paso.

A diferencia del patrón Factory, el Builder permite crear objetos configurables, con múltiples opciones o pasos opcionales.

Builder

```
// Método  =>       Si el builder crea un String .toString()
//                  Si el builder crea un objeto .build()

// Ejemplo   => StringBuilder
StringBuilder sb = new StringBuilder();
sb.append("Hola")
  .append(" ")
  .append("Mundo")
  .append("!")
  .append(" Cómo estás?");

// Crear String desde el StringBuilder
String result = sb.toString();
```

3.5 Parser Factory

Combina el disponer de un PARSER con el crear un objeto mediante un patrón de tipo FACTORY.

3.6 Parser Builder

Combina el disponer de un PARSER con el crear un objeto mediante un patrón de tipo BUILDER.